Systems Design

Stock Trading App

Stephen Colonna, William Persigehl, Isaac Weakland

Professor Walsh

September 27, 2026

Table of Contents

Background 3
Problem Statement 3
Technology Solution 3
Process Maps 4
Requirements 13
User Interface 17
Wireframe 1: Customer Trading Dashboard 17
Description 17
Wireframe 2: Administrator Stock Creation Form 18
Description 18
Information Architecture 19
Entity Relationship Diagram 19
Infrastructure Architecture 21
Network Topology 21
Security and Privacy Architecture 25
Programming 27
References 29

Background

Problem Statement

The company lacks a safe, hands-on environment for new stockbrokers to practice trading without exposing the firm to financial risk. Currently, new stockbrokers have no simulated environment to practice trading. Learning on live markets risks costly errors, and classroom-only training fails to build practical skills in placing orders, settling transactions, or managing portfolios and cash accounts. The firm requires a realistic practice environment that behaves like a real market but carries zero financial risk.

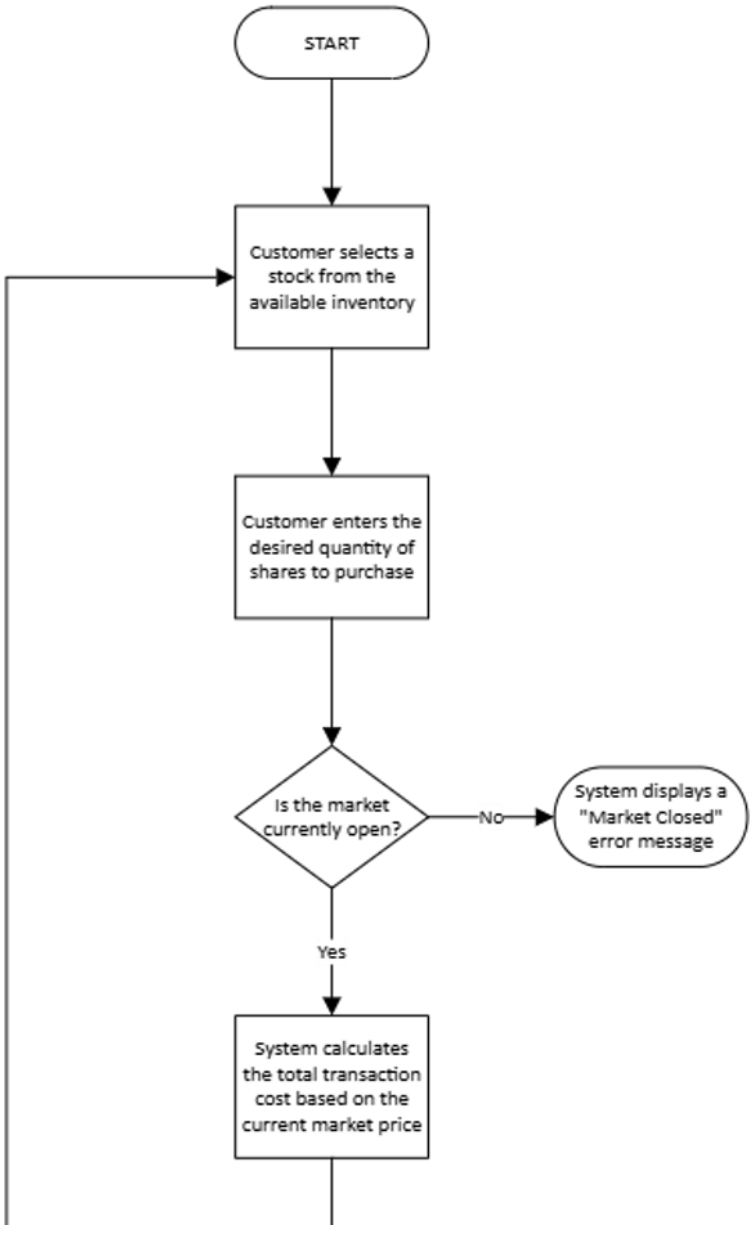
Technology Solution

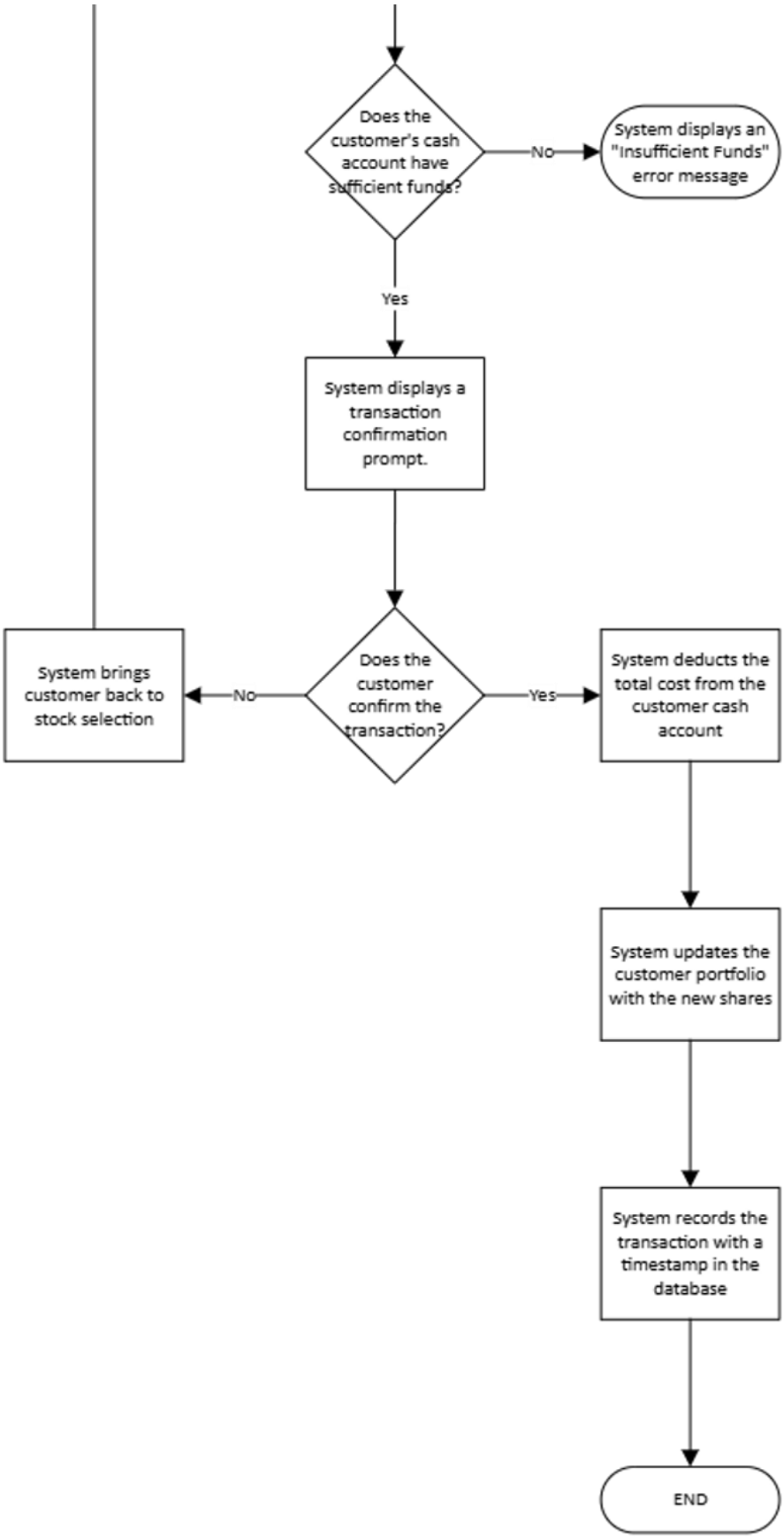
The proposed solution is the design and development of an in-house Stock Trading System configured as a 3-tier web application. This web application will feature a browser-based user interface for trading, a Flask application server hosted on AWS EC2 for business logic (Amazon, 2026a), and a MySQL database on AWS RDS for secure data storage (MySQL, 2024). Customers will interact with simulated stocks whose prices fluctuate during market hours, while administrators will control stock creation and market schedules.

Process Maps

Function: This process map illustrates the steps required for a customer to purchase shares of a stock from the inventory.

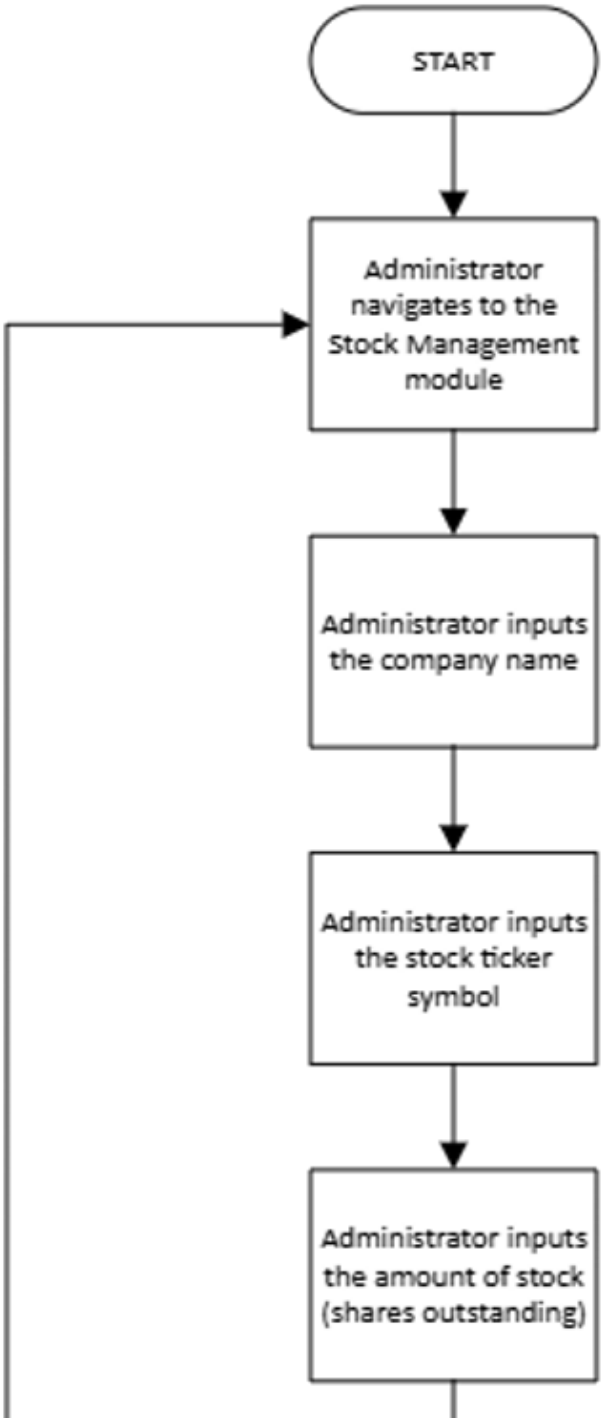
Process Map #1: Customer Buy Stock

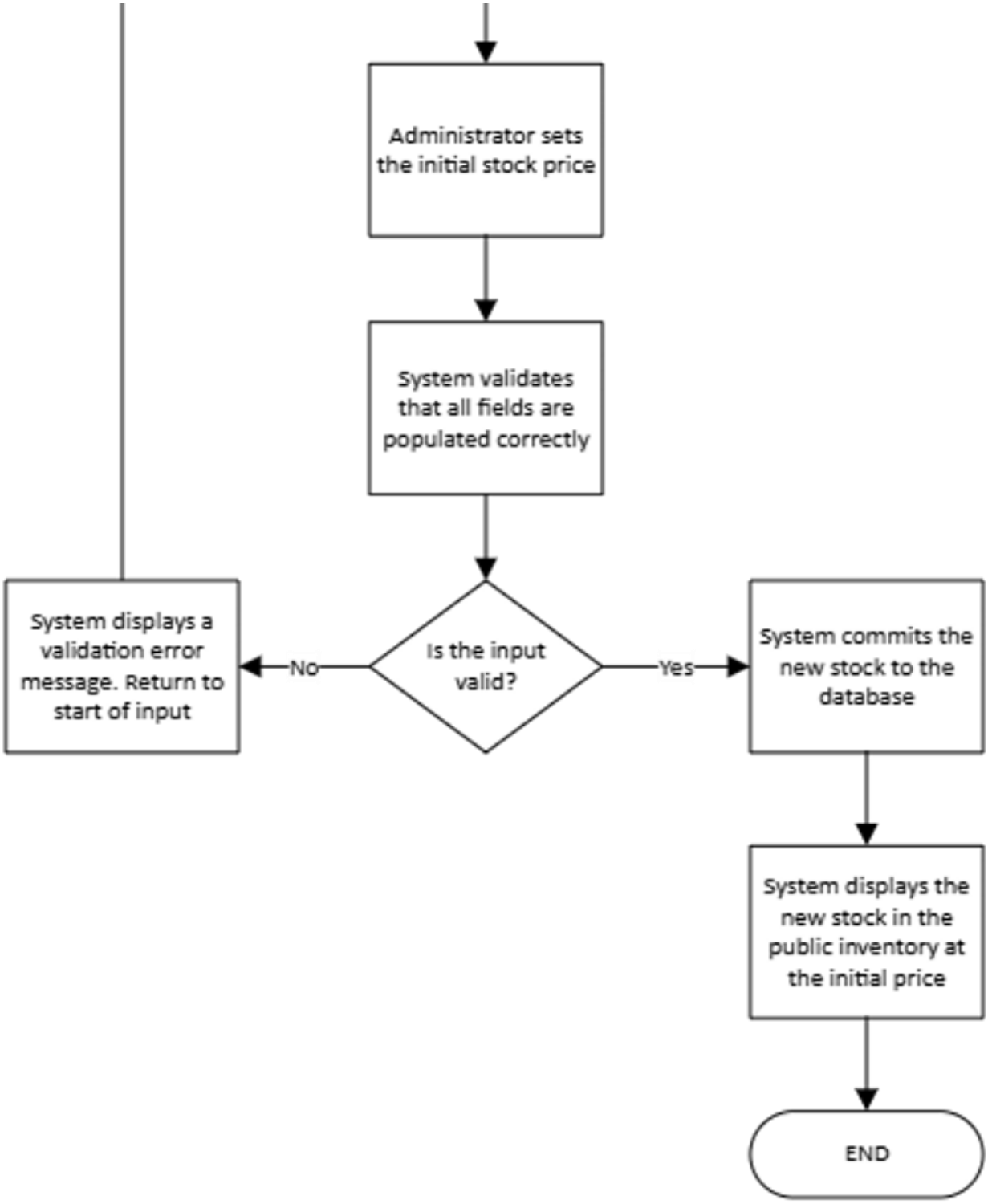



Does the customer's cash account have sufficient funds?
No
System displays an "Insufficient Funds" error message
Yes
System displays a transaction confirmation prompt.
Does the customer confirm the transaction?
No
System brings customer back to stock selection
Yes
System deducts the total cost from the customer cash account
System updates the customer portfolio with the new shares
System records the transaction with a timestamp in the database
END

Function: This process map defines the workflow for an administrator adding a new stock to the platform's market inventory.

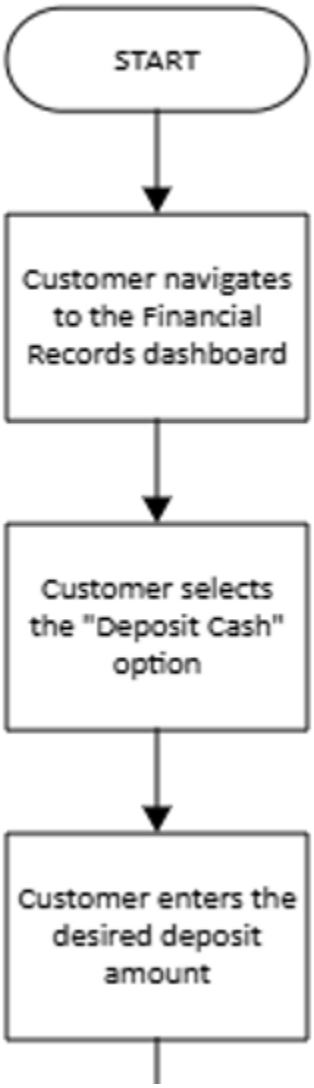
Process Map #2: Administrator Create Stock

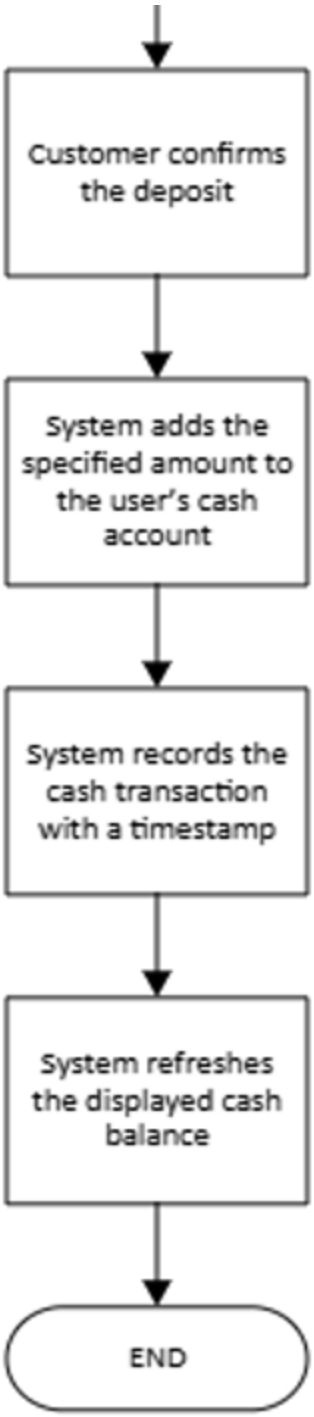



Administrator sets the initial stock price
System validates that all fields are populated correctly
Is the input valid?
No
Yes
System displays a validation error message. Return to start of input
System commits the new stock to the database
System displays the new stock in the public inventory at the initial price
END

Function: This process map outlines the steps for a customer to add simulated funds to their account.

Process Map #3: Customer Deposit Cash




Customer confirms the deposit
System adds the specified amount to the user's cash account
System records the cash transaction with a timestamp
System refreshes the displayed cash balance
END

Corrections:

- **Error Message Shapes:** In Process Map #1 (Customer Buy Stock), change the "Market Closed" and "Insufficient Funds" error displays from ovals to rectangles. Route the "No" paths from these error checks to loop back to stock selection or terminate cleanly through a designated error-handling path rather than improper terminal ovals.
- **Deposit Cash Validation & Consistency:** In Process Map #3 (Customer Deposit Cash), insert a decision diamond immediately after "Customer enters the desired deposit amount" labeled "Is the deposit amount valid?". Add a "No" branch pointing to a rectangle displaying a validation error message that returns the customer to the deposit entry step. Update the confirmation step to a decision diamond (Does the customer confirm the deposit?) with a "No" branch returning the user to the dashboard, matching the consistency of the Buy Stock map.

Process Map #4: User Login

- *START* → User navigates to login page → User enters username and password → System validates credentials → **[Decision] Are credentials valid?** → *No:* System displays login error message → return to login entry. → *Yes:* System checks user role → **[Decision] Is user an Administrator?** → *Yes:* Route to Administrator Dashboard → *No:* Route to Customer Trading Dashboard → *END*.

Process Map #5: Customer Sell Stock

- *START* → Customer selects owned stock from portfolio → Customer enters desired quantity of shares to sell → **[Decision] Is the market currently open?** → *No:* System displays "Market Closed" error → *Yes:* **[Decision] Does customer own sufficient shares?** → *No:* System displays "Insufficient Shares" error → *Yes:* System calculates total sale proceeds based on current market price → System displays transaction confirmation prompt → **[Decision] Does customer confirm sale?** → *No:* Return to portfolio → *Yes:* System credits proceeds to customer cash account → System removes shares from user portfolio and returns them to inventory → System records transaction with timestamp → *END*.

Process Map #6: Customer Withdraw Cash

- *START* → Customer navigates to Financial Records / Withdraw dashboard → Customer enters desired withdrawal amount → **[Decision] Does account have sufficient available balance?** → *No:* System displays "Insufficient Funds" error → *Yes:* System displays withdrawal confirmation prompt → **[Decision] Does customer confirm withdrawal?** → *No:* Return to dashboard → *Yes:* System deducts amount from cash account → System records cash transaction with timestamp → System refreshes displayed cash balance → *END*.

Process Map #7: Administrator Change Market Hours

- *START* → Administrator navigates to Market Control module → Administrator modifies opening time, closing time, or operating days → System validates schedule inputs → **[Decision] Are schedule parameters valid?** → *No:* System displays validation error → *Yes:* System updates market schedule in database → System confirms schedule update → *END*.

Process Map #8: Random Stock Price Generator (System Background Process)

- *START* (Triggered by market timer interval during open hours) → System queries active stocks → For each stock, system generates a random price fluctuation delta → System updates stock current price in database → System records price update timestamp → **[Decision] Is market still open?** → *Yes:* Wait for next timer interval → *No:* Halt price fluctuation until market reopens → *END*.

Requirements

The requirements below define the characteristics and features of the desired system. They avoid implementation bias by describing what the system will do rather than how it will be implemented; each requirement is atomic and measurable. They are split into three separate sections; Customer Requirements, Administrator Requirements, and System Requirements.

Customer Requirements

- The system will execute stock purchases at the current market price.
- The system will execute stock sales at the current market price.
- The system will deduct the cost of stock purchases from the user’s cash account.
- The system will generate a confirmation prompt before a transaction is fulfilled.
- The system will validate that sufficient funds exist in the cash account prior to purchase execution.
- The system will return shares to the stock inventory upon a successful stock sale.
- The system will update the user portfolio immediately after each transaction.
- The system will update the user order history immediately after each transaction.
- The system will update financial records immediately after each transaction.
- The system will display the current user's portfolio of stocks.
- The system will display the current user's cash balance.
- The system will display the user's history of transactions.
- The system will process cash deposits initiated by the user.
- The system will process cash withdrawals initiated by the user.

- The system will credit deposited funds to the user's cash account.
- The system will restrict cash withdrawals to the available balance in the user's cash account.
- The system will deposit the proceeds from stock sales into the user's cash account.
- The system will require each user to log in securely with valid credentials.
- The system will route authenticated users to their respective role-based dashboard (Customer Trading Dashboard or Administrator Dashboard).
- The system will allow the customer to select a stock from the available market inventory.
- The system will allow the customer to enter the desired quantity of shares for trade execution.
- The system will calculate and display the total cost/proceeds of a proposed stock transaction.
- The system will allow the customer to navigate to the Financial Records dashboard to view balances and transaction histories.
- The system will allow the customer to initiate cash deposits and cash withdrawals.

Administrator Requirements

- The system will allow the administrator to modify the stock inventory.
- The system will allow the administrator to create new stocks.
- The system will require the administrator to provide a company name during stock creation.
- The system will require the administrator to provide a stock ticker during stock creation.
- The system will require the administrator to define the shares outstanding during stock creation.
- The system will require the administrator to set the initial price during stock creation.
- The system will restrict the administrator from altering a stock price after initial creation.
- The system will allow the administrator to modify the market hours.
- The system will allow the administrator to input an optional stock description during stock creation.
- The system will allow the administrator to upload a company logo image during stock creation.
- The system will allow the administrator to toggle a Manual Market Override to open or close the market on demand.
- The system will validate administrator input during stock creation and display validation error messages upon invalid entries.

System Requirements

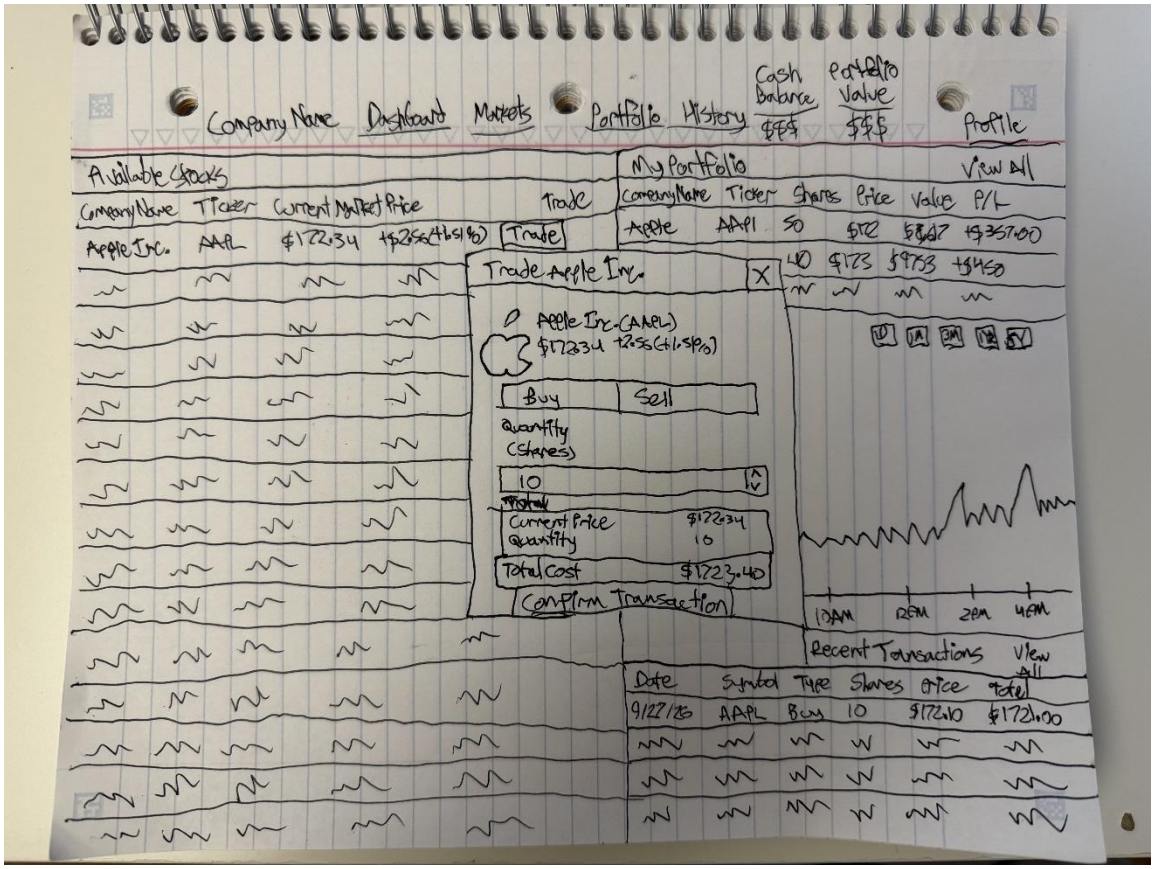
- The system will generate a random market price for each stock.
- The system will fluctuate stock prices gradually during open market hours.
- The system will maintain static stock prices while the market is closed.
- The system will display the current market price for all existing stocks.
- The system will reject trade execution requests submitted outside of established market hours.
- The system will execute valid orders only after user confirmation.
- The system will prohibit the cancellation of confirmed transactions.
- The system will record a timestamp for all account creation events.
- The system will record a timestamp for all stock transactions.
- The system will record a timestamp for all cash transactions.
- The system will display a "Market Closed" error message when trade requests are submitted outside operating hours.
- The system will display an "Insufficient Funds" error message when purchase or withdrawal requests exceed available cash balances.
- The system will allow new users to create an account within the system.

User Interface

Wireframe 1: Customer Trading Dashboard

Description

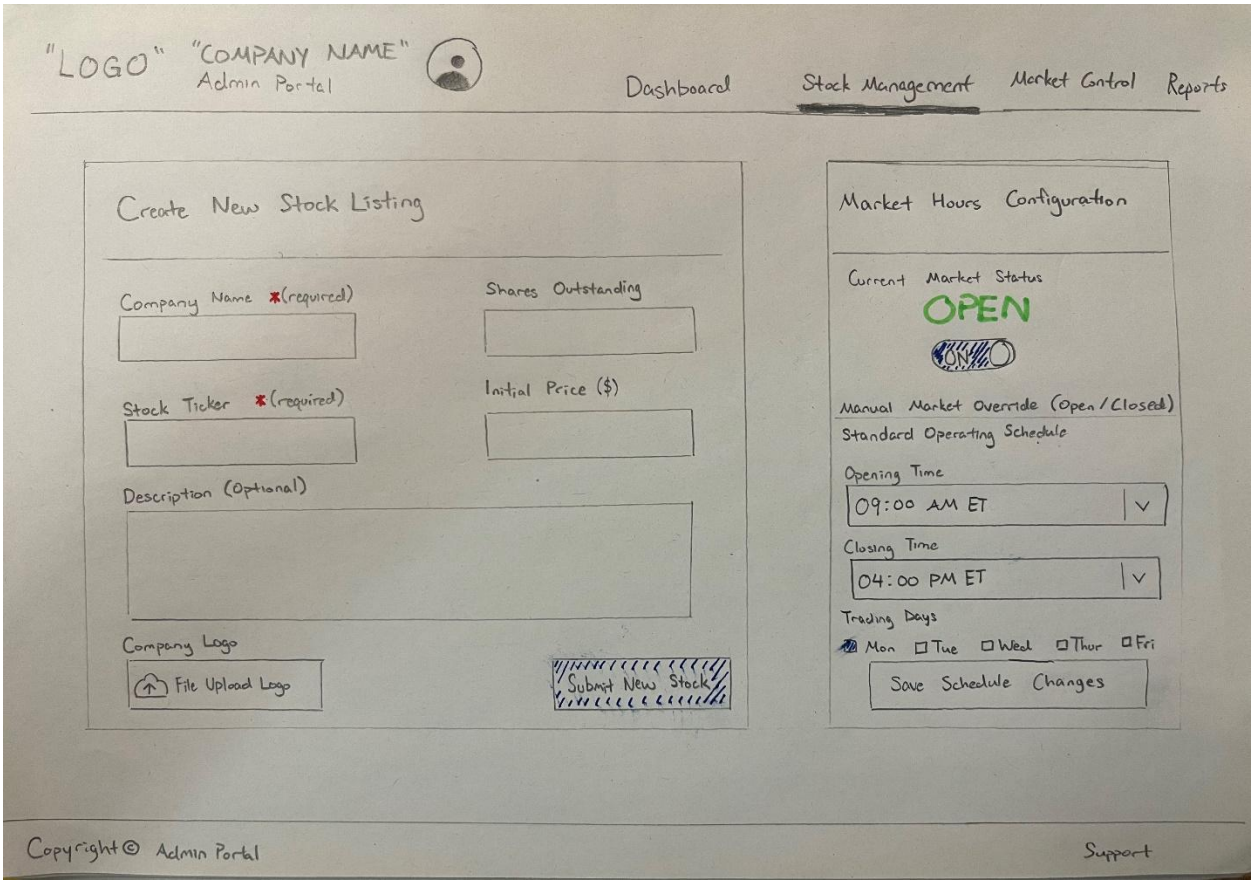
This mockup aligns with the Customer Buy Stock process map. The interface features a top navigation bar displaying the customer's current cash balance and portfolio value. The main content area is split into two sections. The left section contains a dynamic table listing all available stocks, displaying the Company Name, Ticker, and Current Market Price. Each row features a "Trade" button. Clicking "Trade" opens a modal window overlay. This modal prompts the user to select "Buy" or "Sell", enter the desired quantity, and view the total calculated cost. A prominent "Confirm Transaction" button is placed at the bottom of the modal to fulfill the confirmation requirement.



Wireframe 2: Administrator Stock Creation Form

Description

This mockup aligns with the Administrator Create Stock process map. The screen displays an administrative dashboard with a clean, structured form in the center. The form contains required text input fields for "Company Name" and "Stock Ticker", alongside numeric input fields for "Shares Outstanding" and "Initial Price". A "Submit New Stock" button is located at the bottom of the form. To the right of the form, a secondary panel displays the current Market Hours settings, featuring a toggle switch to manually open or close the market and fields to adjust the standard operating schedule.

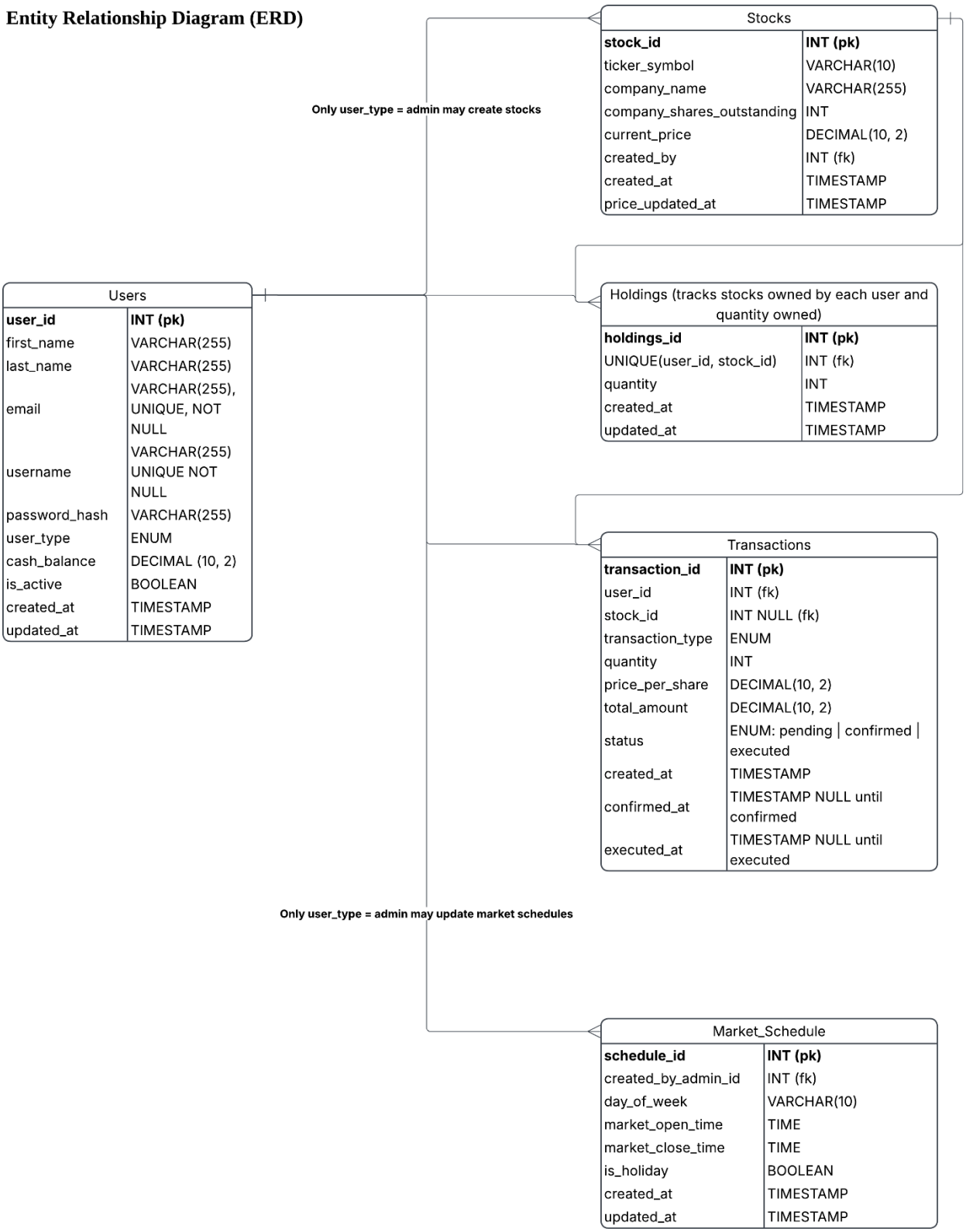


Information Architecture

Entity Relationship Diagram

The information architecture utilizes an Entity-Relationship Diagram (ERD) to support the requirements and establish how data is structured. Making a conceptual ERD model assists the designers and the business in seeing the relationships in the data.

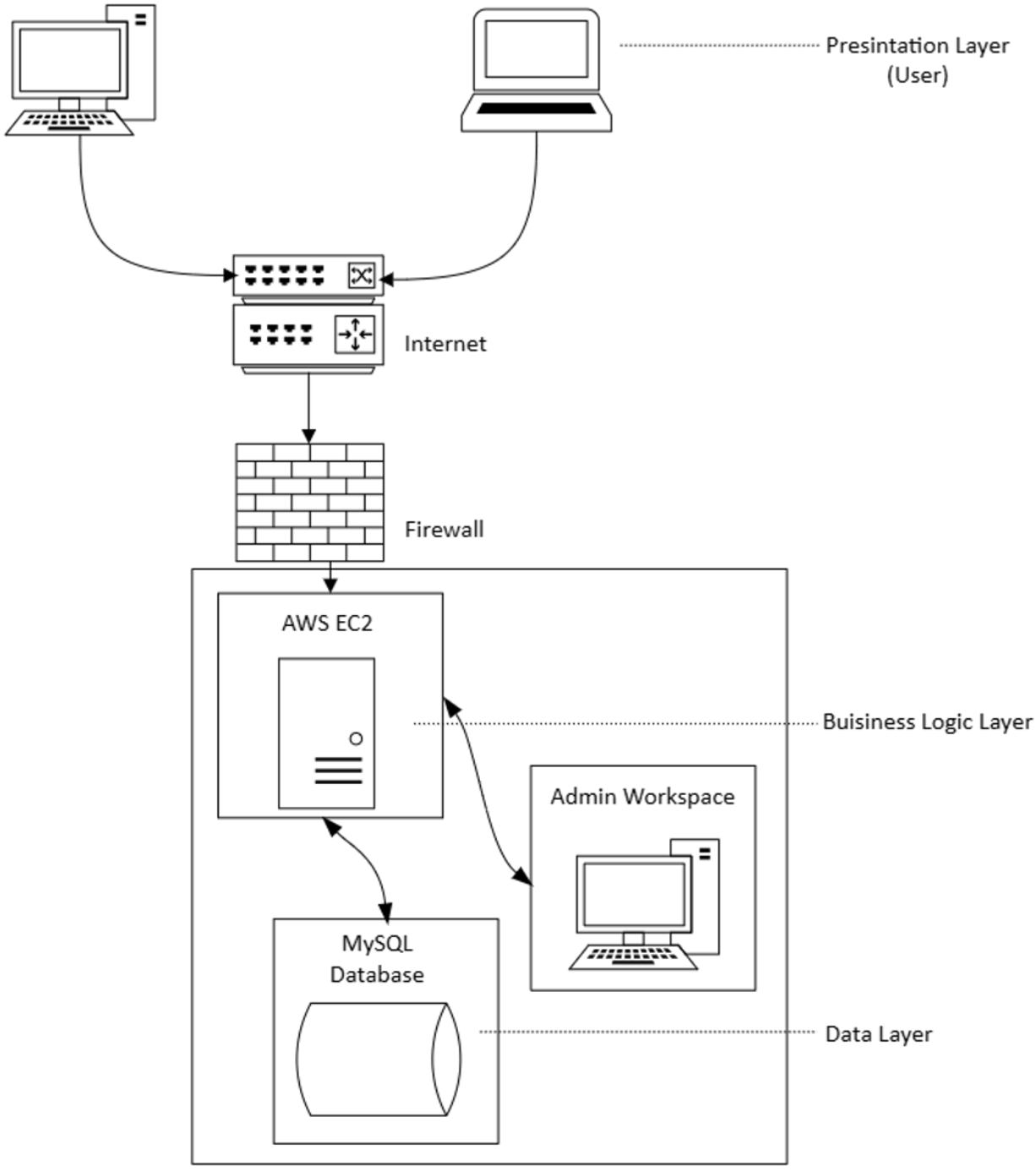
Entity Relationship Diagram (ERD)



Infrastructure Architecture

Network Topology

Infrastructure Diagram



The Stock Trading System is a web-based client/server application implemented using a 3-tier architecture entirely within the AWS Cloud environment. Web applications are platform-independent and use the Internet as a communication backbone while using Hypertext Transport Protocol (HTTP).

Network Components & Data Flow

- **Client & Internet Layer:** End-users (trainees and administrators) access the application via web browsers across the public internet.
- **Firewall:** All incoming web traffic from the internet passes through a network security firewall configured on AWS to inspect incoming packets, filter malicious traffic, and restrict unauthorized access to backend application servers.
- **Admin Workspace:** Positioned securely within the cloud boundary alongside the application tier, the Admin Workspace represents authorized management terminals or secure management consoles. Administrators connect to this workspace via encrypted administrative tunnels (such as VPN or SSH over HTTPS) to access management modules, configure market hours, and oversee platform operations.

3-Tier Logical Architecture

- **Tier 1: Presentation Layer:** Hosted dynamically by the application server, delivering HTML, CSS, and JavaScript directly to the user's browser.
    - *Location:* User's browser or client.
    - *Function:* Focused on the user interface and overall user experience.
    - *Components:* HTML templates, CSS styling, and JavaScript interactivity served by the backend.
    - *Access:* Designed to allow public internet users to interact with the system directly through their web browser. The browser never accesses the database directly.

- **Tier 2: Application Layer (Business Logic Layer):** Hosted on an AWS EC2 instance within a public subnet, running the Flask web application server and core business logic.
    - *Location:* AWS EC2 instance located within a public subnet (Amazon, 2026a).
    - *Function:* Handles the business logic, processes incoming user requests, and manages application routing.
    - *Components:* Flask web framework, underlying Python code, and internal API endpoints.
    - *Responsibilities:* Processes HTTP requests, enforces business rules (such as validating available funds and enforcing market hours), communicates with the database tier, and generates responses to the client.
    - *Port Configuration:* Exposed on port 8000 to the public internet.
- **Tier 3: Data Layer:** Hosted on a MySQL database instance within an isolated AWS RDS private subnet, accessible exclusively by the EC2 application server over port 3306.
    - *Location:* MySQL relational database hosted on an AWS RDS instance within a private subnet.
    - *Function:* Handles the secure storage and management of all system and user data.
    - *Components:* MySQL database.
    - *Security and Access:* Fully isolated in the private subnet with no direct internet access. It is accessible only from the application tier via port 3306.

Security and Privacy Architecture

The system must protect organizational and user data across three core security principles: Confidentiality, Integrity, and Availability.

1. **Confidentiality:**
    - *Risk:* Unauthorized public access to sensitive user financial data and transaction histories.
    - *Control:* The system implements a secure login function to ensure only authenticated users can access portfolios. Furthermore, the MySQL database is isolated within a private subnet on AWS RDS, ensuring it has no direct internet access and can only be queried by the authorized application tier.
2. **Integrity:**
    - *Risk:* Users attempting to manipulate market prices, alter past financial records, or cancel trades after-market execution.
    - *Control:* The business logic explicitly enforces that once a transaction is confirmed by the user, it cannot be cancelled. Additionally, the system data architecture requires event logging, ensuring that timestamps for account creation, stock transactions, and cash transactions are securely recorded to create an immutable audit trail.

3. **Availability:**
    - *Risk:* System downtime during active market hours, preventing trainees from executing necessary trades.
    - *Control:* The system leverages the AWS Cloud environment, utilizing an EC2 instance for the application server and an RDS instance for the database (Amazon, 2026b). This infrastructure mitigates downtime risks by relying on established cloud hardware rather than local desktop applications confined to standalone machines.

Programming

The development process will utilize specific tools and programming languages to build the front-end presentation and back-end logic of the web application.

Development Tools

- **AWS EC2 and RDS:** Selected to host the application and database, offering low-cost cloud hosting and zero per-user commercial license fees (Amazon, 2026a), (Amazon, 2026b).
- **Git (and GitHub):** Utilized for version control. Git commands such as init, add, and commit are necessary for managing code versions and collaborating effectively within the development team (Git, 2019).
- **Text Editors / IDEs:** Tools like Visual Studio Code, Notepad++, or Vim will be used by the development team to utilize shortcuts, syntax highlighting, and integrated development facilities.
- **Database Solution:** MySQL will be chosen as the relational database server to organize data in the form of tables and efficiently retrieve information related to system queries (MySQL, 2024).

Programming Languages

- **Python:** Selected for the back-end application logic. Python is a programming language that lets developers work quickly and integrate systems more efficiently. It natively supports the Flask web framework required by the business case to process HTTP requests and enforce trading rules (Pallets, 2024).
- **HTML (Hyper Text Markup Language):** Selected to design the front-end portion of the web pages. HTML is a markup language used to define the text documentation within tags and establish the structure of the web pages.
- **CSS (Cascading Style Sheets):** Selected to apply styles to the web pages independently of the HTML. CSS simplifies the process of making web pages presentable and user-friendly.
- **JavaScript:** Selected to make the site interactive for the user. JavaScript will be utilized on the client side to handle user events, such as dynamic calculations within the transaction confirmation modal before data is sent to the Flask server.
- **SQL (Structured Query Language):** Selected as the standard database language used to create, maintain, and retrieve the relational database hosted on the RDS instance.

References

Amazon. (2026a). *Amazon EC2 documentation*. Amazon Web Services, Inc. https://aws.amazon.com/documentation-overview/ec2/

Amazon. (2026b). *Amazon Relational Database Service documentation*. Amazon Web Services, Inc. https://aws.amazon.com/documentation-overview/relational-database-service/

Git. (2019). *Git - about version control*. scm. Git-Scm.Com. https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

MySQL. (2024). *MySQL :: MySQL 8.4 reference manual :: 7.3 The MySQL system schema*. Mysql.Com. https://dev.mysql.com/doc/refman/8.4/en/system-schema.html

Pallets. (2024). *Quickstart — Flask documentation (3.0.x)*. Palletsprojects.Com. https://flask.palletsprojects.com/en/stable/quickstart/